

Sonrisa homework

Req: We want users to be able to set up alerts so they get notified when something important happens in the world — like breaking news, market movements, natural disasters, that kind of thing. Should work for both email and Slack. Make it flexible enough that we can add more channels later. We need an admin view too.

Thoughts:

- in my opinion, for this task, I have to provide a UI for users to ask for alerts, mock SSE to continuously deliver notifications, implement a worker (which also better be mocked for now) which often gets the latest events, connect the solution to Slack and email
- I assume channel means email and Slack at the moment
- I will choose Vite + React + JavaScript for the solution
- I'm using Vite, because it's modern and fast
- I would like to only mock the backend part that this project might require (like getting the event data and no real Slack or Resend integration)
- will have to sort them out based on the user's preferences (i would like to list all the topics that are currently available), probably an AI agent could decide on the level of importance of each event
- I think it would be a good idea for the user to be able to add some keywords of what kind of events matter the most, like earthquake above 7, and feed the input to the LLM
- for the admin part I would add settings like which AI model to use, which APIs to include as option
- since there are many missing details about this project, the first thing I will do is to have a grilling session with Codex, so we can be on the same page of ideas
- I'm not familiar with how do I integrate this notification system in Slack, but luckily there is AI to make things clearer

Plan:

- since speed, efficiency and quality are equally important for the company, I will start off by creating the project using Vite, then using the Codex desktop app, starting a grill-me session about the requirements and my personal thoughts
 - after grill-me, I will turn the session into a prd, then turn them into tickets (i prefer local md files for this challenge)
 - then i'll use /implement on the tickets, and lastly /code review
- this is the workflow I currently like the most :)

My workflow steps:

- once I created the project, I'm adding it to GitHub
- I'm using 5.6 Luna with medium effort because I found it great in term of value per token usage when implementing not very complicated stuff
- I'm having a grill-me session

? Q11 — Which process artifacts should be part of the submission?

Possible artifacts include:

- A. Product brief, decision log, implementation plan, prompt drafts, validation checklist, and final reflection
- B. Only a README describing the implementation
- C. A full chronological transcript of every AI interaction

👉 I recommend A. Curate the important reasoning instead of dumping the entire transcript. The evaluator wants evidence of judgment, not volume.

? Q12 — Where should the process artifacts live?

- A. A documented `docs/` folder in the repository
- B. Only in the task/chat history
- C. Both: curated files in `docs/`, with the conversation as supporting evidence

👉 I recommend C. The repository should stand on its own, while the conversation demonstrates how you directed the AI.

? Q13 — What is the alert creation experience?

- A. A structured form: topic, conditions, severity, destination ↓ and enabled/disabled state
- B. A natural-language input that generates the structured form

C

Q8: A. because it will look more like a real application

Q11: I'm already filling a document with steps I make and also added the thoughts I showed you, once I'll have a plan markdown I will send it separately, and I'll probably ask after chat to write the whole conversation into a file. I like the idea of curating the most important reasoning behind the decisions, it provides a nice overview besides reading the whole grill me session. Add any other artifacts that you consider useful

Q12:

+ 🗨 Ask for approval

GPT-5.6 Luna Medium



- at Q12 I realized that even the chat history will automatically be added in a folder if I say so, so I'll delete "and I'll probably ask after chat to write the whole conversation into a file" and change "Add any other artifacts that you consider useful" into "Add the artifacts you have mentioned", to surely avoid leaving out saving chat history
- I asked this in the process: I have one more question: the task specifically mentions that notify if something important happens - is there any other way of evaluating importance than needing an LLM to evaluate by itself or by user input? then I realized that this is what we were talking about, basically including some data for types of event as a guideline of event severity
- called /to-spec skill to turn everything we discussed into a [spec.md](#)
- grilling is great because you clarify so many details where the other way the implementation could go wrong in so many directions, so the plan that results from grilling is usually very well specified
- called /to-tickets and after I accepted the task splitting it created the tickets that do not have to be executed linearly because each one contains blocking factors, so some of them can run in parallel by multiple agents
- keeping in mind that context windows have "smart zones", since I have the task tickets, I can now initialize a new chat to reset the context

- then i call /implement on the folder where tickets are located and I might mention to use ponytail skill that I recently discovered, it is great for simplifying code and it really thinks like a senior developer trying to catch duplicated logic, unnecessary steps etc.
- I did not add ask for tests on purpose, I would like to discover bugs myself for the sake of homework but I also asked the AI to use /agent-browser which was not installed in this session apparently, but it used other feature that was similar
- I'm not running /code-review this time, because I asked the AI to use ponytail explicitly after each change
- I remembered that I asked some more changes so the docs need an update
- I forgot to take care of major commits, I was so involved in the process, so I separated the changes in chunks afterwards
- I evaluated the code carefully before committing it